

Programmation 1 – Épreuve finale numéro 1

Durée : 1 heure.

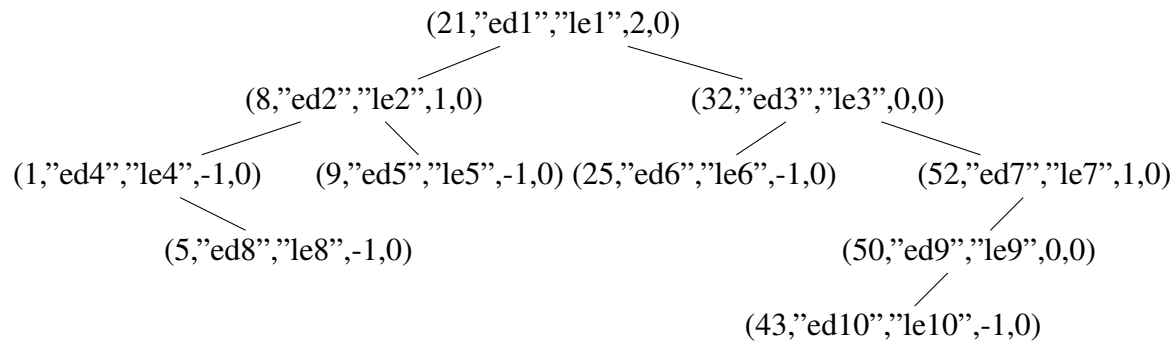
Les documents manuscrits et dactylographiés sont autorisés. Les exercices sont indépendants. La clarté et la précision de la rédaction (en particulier les commentaires du code) seront un facteur important de l'évaluation. Le barème est donné à titre indicatif.

Nous avons les classes suivantes :

```
1  /**
2   * La classe représentant des cartes postales. La fonction equals
3   * compare seulement les postcardID.
4   */
5  public class Postcard {
6      public int postcardID;        // identifiant de la carte
7      public String editor;        // éditeur de la carte
8      public String legend;        // légende de la carte
9      public int leftHeight;       // la hauteur du sous-arbre gauche
10                                     // par convention, un butoir a une hauteur de -1
11      public int extraField;       // champ supplémentaire
12      ...
13      public boolean equals(Object o) { ... }
14      ...
15  }
```

```
1  /**
2   * Les cartes postales sont organisés pour une recherche symétrique
3   * (comme un arbre binaire de recherche) relativement à l'attribut
4   * postcardID.
5   * Par définition, tous les identifiants sont différents.
6   *
7   * On suppose que les champs leftHeight sont déjà remplis.
8   */
9  public class PostcardCollection extends BinaryTree<Postcard>
```

Écrire les méthodes d'instance suivantes de la classe `PostcardCollection` :



Question 1 (2 pts)

```

1  /**
2   * @param postcardID    identifiant de carte postale
3   * @return    true si une carte d'identifiant postcardID est
4   *           présente dans this, false sinon
5   */
6  public boolean findPostcard(int postcardID)

```

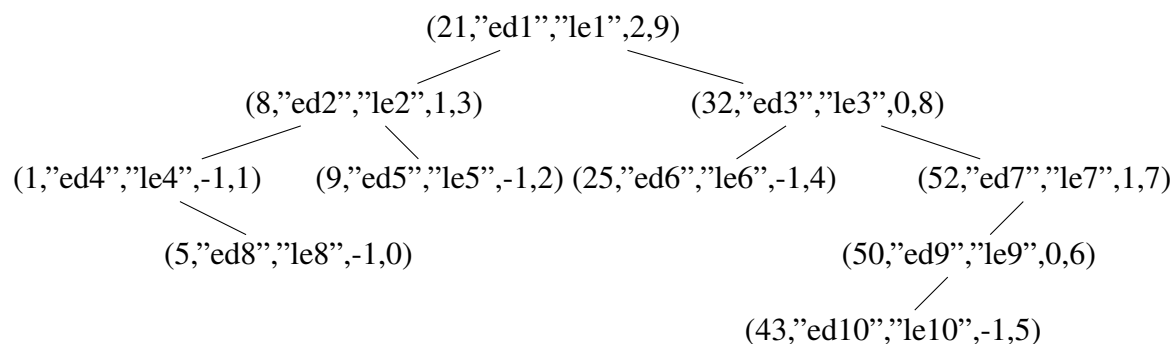
Question 2 (4 pts)

```

1  /**
2   * Stocker l'ordre de parcours postfixe de l'arbre this dans le
3   *   champ extraField.
4   * Indication : on peut créer un attribut d'instance supplémentaire
5   *   dans la classe PostcardCollection.
6   */
7  public void addPostOrderNumbers()

```

Pour l'arbre de l'énoncé, cela doit donner :



Question 3 (4 pts)

```
1  /**
2   * @return la hauteur de l'arbre, calculée de façon efficace
3   */
4  public int height()
```

Question 4 (5 pts)

```
1  /**
2   * @pre !this.isEmpty()
3   * @return la légende la plus longue parmi les cartes postales
4   *          présentes dans this
5   *          (choisir une d'elles dans le cas d'ex-aequo)
6   */
7  public String longestLegend()
```

Question 5 (5 pts)

```
1  /**
2   * @return true si pour chaque nœud la différence entre les nombres
3   *          de nœuds des deux sous-arbres est inférieure ou égale à 1,
4   *          false sinon
5   *
6   * Indication : il peut être nécessaire de créer une classe interne.
7   */
8  public boolean isNodeBalancedTree()
```